

Linearizable KV Store

Part 1: KV Server with Reliable Network

The server maintains an in-memory map where each key stores a (value, version) pair. The version number tracks how many times a key has been written.

Two RPC handlers were implemented:

- Get: looks up a key and returns its value and version. Returns ErrNoKey if the key doesn't exist.
- Put: A conditional write. The client must supply the version it expects the key to currently have. The server only applies the update if the versions match and then increments the version. If the versions don't match, it returns ErrVersion. A client creates a brand new key by sending version 0.

On the client side, the Clerk.Get and Clerk.Put methods were wired up to send these RPCs to the server.

The version number is what makes the server linearizable. Because every write is conditioned on the current version, two clients racing to update the same key can't silently overwrite each other. This means only one wins, and the other gets ErrVersion and knows it needs to re-read and retry. Every read is also guaranteed to reflect all writes that were completed before it started.

Part 2: Mutex

From part 1, because we know that our KV store is linearizable, it means that only one action can be performed on any given entry at a time. This means that if an entry is changed by one client, that change will appear for the actions of all other clients. Thus, if a client makes a change to one entry, another client cannot make a change to the entry with the same version number as the first change. We can use this property to create a mutex lock, with acquire() and release().

- Lock Creation: We take a string, and create a DB entry with that as the key, with the value being a string. If the lock is currently held, that string is set to "held", otherwise it's set to "available".
- Acquire: We wrap all functionality in an infinite loop, so the acquire function continuously tries to acquire the lock until it actually does. Inside this loop, we first get the version number of the current entry, then if the lock DB entry is not "held", we attempt to acquire it with that version number. If another client has acquired the lock, then the version numbers will not match and the DB response will fail, guaranteeing that only one client can hold the lock at a time. If the lock is acquired, we set the lock's internal held state to true and set the version number to the new one in the database.
- Release: If the lock's internal state is held, then we set it to false and update the database as such. The version number should directly match the database, because the only way the state in the database would change with an value of "held" was if release was successfully called, and only this lock is able to successfully call release.

Part 3: Unreliable Network

So far, we have assumed a reliable network. This is not the case in real life however, as RPC's can be dropped before they reach the target receiver due to networks being unreliable. In the context of our KV store, the client sends Put or Get requests to the server, and the server responds with a reply of either Ok, ErrNoKey, or ErrVersion (As explained before). If the client's request RPC gets dropped, there really are not any complications that occur—the client can just send its request again after a timeout. However, if the server's response is dropped, an issue may occur in the following sequence of events:

1. The client sends a Put request, which the server executes successfully.

2. The server has executed the Put request, but the server's response is dropped by the network so the client does not know if the result was successful or not. Should the client try again?
3. If it does, it will receive an ErrVersion response, even though the Put request had already gone through successfully previously.

At this specific moment in time, the client's understanding of the server's state is incorrect, which can lead to issues like deadlock if you try to implement a lock with this flawed KV store (since the client holds the lock to the Put request object, but is not aware of this).

So how do we adapt our KV store to be able to handle unreliable networks gracefully? The clerk middleware will handle Get/Put requests and any server responses in the following manner:

- Get requests
 - Just make the RPC repeatedly in a loop with a delay until you get a reply. Since Get is read-only/idempotent, any server response is the correct answer.
- Put requests
 - Same retry loop as the Get request, but add a "retried" boolean/flag that starts false and flips to true after the first attempt gets no reply.
 - Then there are a few cases:
 - First attempt receives a response from the server - Great! Whatever the server responded with is correct
 - On the 2nd and subsequent retry attempts:
 - Ok/ErrNoKey response from the server - the retry was successful and trust the response
 - ErrVersion - AMBIGUOUS, we do not know whether the request was executed or not. Return an ErrMaybe response to the client

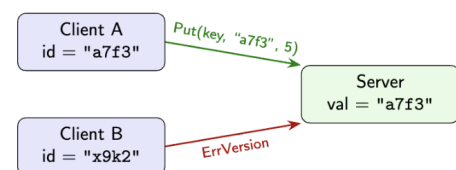
Part 4: Mutex with network failure

Part 4 extends the Part 2 locking mechanism to work correctly when the network can drop RPCs. The main challenge is handling the ErrMaybe return value from Put, which essentially means "this operation might or might not have been applied."

In Part 2, the lock stored either *"available"* or *"held"* as the value. The Acquire function sends a Get, then tries to Put *"held"* using the version. On a reliable network, Put only returns an OK or ErrVersion. However, on an unreliable network, Put can also return ErrMaybe. This happens when the client's RPC request went through, the server applied it, but the reply got lost. The Clerk retries, the server says ErrVersion because the version has already advanced, and the Clerk returns ErrMaybe to indicate the ambiguous state.

Problems with old approach:

- (1) **Deadlock.** The old Acquire code only checked for `err == rpc.OK`. If it got anything else, it fell through and looped. When ErrMaybe came back, the code looped back to Get, saw the lock value was *"held"* (because the Put actually succeeded on the server), and since the old check was `if value != "held"`, it skipped the Put and kept spinning. The client held the lock but didn't know it. No one would ever release it, and every client in the system would deadlock.
- (2) **Broken Mutual exclusion:** A naive fix is to check the version after ErrMaybe: if it is incremented by one, assume your Put went through. But this breaks with concurrent clients. If A and B both read version 5, A Puts *"held"* and gets ErrMaybe while B's Put succeeds (version → 6), A then sees *"held"* at version 6 and wrongly claims the lock. Causing both clients to think they hold the lock.



After ErrMaybe, Get and check:

Solution: Each Lock gets a unique random ID. **Acquire** writes this ID instead of "held", so now the lock has three states: "available" (free), our own ID (we hold it), or any other ID (someone else holds it). After ErrMaybe, we **Get** and check, if the value is our ID, our Put went through; else we don't hold it and back off.

Part 5: Extension with the help of Claude for implementation (see branch raft-byzantine for code)

Our extension builds off of Raft, where it is assumed that all nodes in the Raft cluster communicate honestly. However, in the real world, malicious actors or compromised nodes can perform Man-in-the-Middle (MITM) attacks to intercept, modify, or spoof RPCs. In the context of our Raft implementation, an attacker could alter the Term in an AppendEntries RPC to force a leader to step down, or modify the entries themselves to corrupt the replicated log. If these messages are tampered with, the fundamental safety properties of Raft (such as leader completeness and state machine safety) are violated, leading to a total breakdown of consensus.

We simulate an adversary by intercepting the RPC byte stream and applying random bit flips.

The process follows this sequence:

1. Every AppendEntries or RequestVote RPC is serialized into a byte array before it hits the network.
2. We define a corruption probability. For every packet, the interceptor decides whether to "attack" it.
3. If an attack is triggered, the interceptor selects a random byte index in the payload and performs a XOR operation with a random bitmask.
4. The corrupted byte array is then sent to the target node.

To adapt our Raft implementation to handle an unreliable, adversarial network described above, we built an HMAC interceptor middleware that handles all RequestVote and AppendEntries RPCs in the following manner:

1. Before the cluster starts, a shared secret key is distributed to all honest nodes. This key is used to generate and verify Hash-based Message Authentication Codes (HMACs) for every packet sent across the wire.
2. For every outgoing request or reply, the interceptor calculates a digest of the entire RPC payload using the shared secret. This signature is appended to the message header. This ensures that any modification to the message body by a MITM will result in a signature mismatch.
3. Upon receiving a packet, the interceptor recalculates the HMAC using the local shared key and compares it to the attached signature.
 - a. Signature Matches: Great! The message is authentic and has not been tampered with. It is passed up to the Raft state machine for processing.
 - b. Signature Mismatches: The packet is immediately dropped. By discarding tampered messages at the interceptor level, the Raft logic never sees the corrupted data, preventing the state machine from transitioning into an inconsistent state.

We wrote tests to observe that:

1. When a malicious actor attempted to increment the Term or modify log entries, the receiver correctly identified the signature mismatch and ignored the packet.
2. Spoofed messages from unauthorized nodes were rejected because the attacker lacked the shared secret required to generate a valid HMAC.

Obviously if a node with the shared secret acts maliciously, our system would fail. To maintain security and consensus under a fully Byzantine scenario, we would have to assign asymmetric key pairs for all nodes.